

30.1 Partitioning

Previously, we've covered **selection sort**, **heap sort**, **merge sort**, and **insertion sort**.

These sorts each have a basic idea powering them:

- Selection sort: find the smallest item, put it at the front
- Heap sort: use MaxPQ to find the maximum element, put it at the front
- Merge sort: merge two sorted halves into one sorted whole
- Insertion sort: determine where to insert the current item

The core idea behind Quicksort involves **partitioning**.

Specifically, to partition an array $a[]$, first pick an element $x = a[i]$ as the pivot.

Partitioning on that pivot element (x) involves rearrange $a[]$ such that:

1. x moves to some position j (can be the original position of i)
2. All array elements to the left of x are less than or equal to x ($\leq x$)
3. All array elements to the right of x are greater than or equal to x ($\geq x$)

30.2 Quicksort Algorithm

The figure below shows a partition performed on the pivot of 5. Notice how 5 in its “proper place” (in other words, it's exactly where it should be if the entire array was sorted).

This suggests that the partition procedure can be used recursively on the two halves to the left and to the right of the pivot element.

The properties that partitioning provides inspires Tony Hoare's Quicksort algorithm.

Quicksort Algorithm

To quicksort N items:

1. Partition on the leftmost item as the pivot.
2. Recursively quicksort the left half.
3. Recursively quicksort the right half.

[Look at the following website for an illustration of Hoare's Quicksort algorithm](#)

30.3 Quicksort Performance Caveats

“Quicksort” was chosen by Tony Hoare as the name for this particular partition sort algorithm. Coincidentally, quicksort is empirically the fastest sort for most common situations.

How fast is Quicksort exactly? To answer this question, we need to count the number and calculate the difficulty of the partition operations employed in Quicksort.

In the theoretical runtime analysis, partitioning costs $\Theta(K)$ time, where $\Theta(K)$ is the number of elements being partitioned.

However, what complicates runtime analysis for Quicksort is that the the overall runtime will depend on where the pivot ends up.

Best Case

For instance, the best case runtime is when the pivot chosen always lands in the middle (in other words, the partition always picks the median element of the array).

In the best case, the total work done at each level is approximately $O(N)$. To see this, the first partition pivots on one element and requires checking all N elements to determine whether they go on the right or the left of the pivot element. The second layer repeats this, but for $N/2$ elements on two separate pivots (since we are recursively partitioning on the two halves). Thus, each level is $O(N)$ work.

The overall runtime becomes $\Theta(NH)$, where $H = \#$ of layers = $\Theta(\log N)$. Thus, the total runtime for Quicksort in the best case is $\Theta(N \log N)$.

Worst Case

The worst case for runtime occurs when the pivot chosen at each partition lands at the beginning of the array. In this scenario, each layer still has to do $O(N)$ work, but now there are $H = \#$ of layers = N layers, since every single element has to be pivoted on. Thus, the worst case runtime is $\Theta(N^2)$.

Quicksort vs Mergesort Performance

Theoretical Analysis Quicksort Mergesort

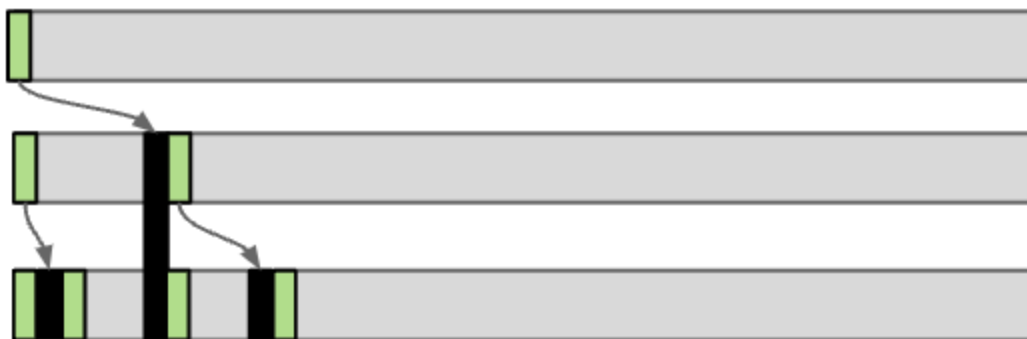
Best Case $\Theta(N \log N)$ $\Theta(N \log N)$

Worst Case $\Theta(N^2)$ $\Theta(N \log N)$

In comparison, Mergesort seems to be a lot better, since Quicksort suffers from a theoretical worst case runtime of $\Theta(N^2)$. So how can Quicksort be the fastest sort empirically? Quicksort's advantage empirically comes from the fact that on average, Quicksort is $\Theta(N \log N)$.

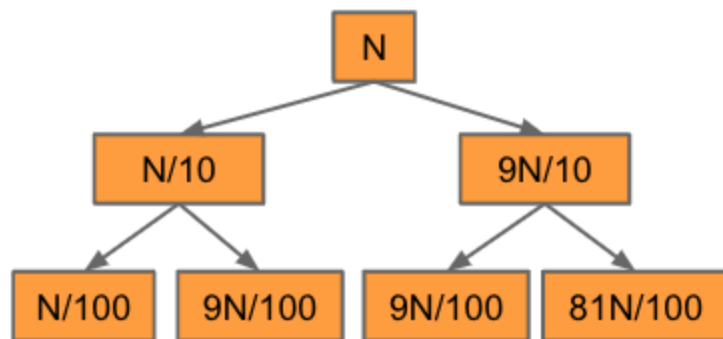
Quicksort's Performance Argument #1: 10% Case

One argument for why Quicksort is the fastest sort empirically supposes that the pivot always ends up at least 10% from either edge.



Pivot is ~10% from the left edge.

If this is the case, then the runtime is still $O(NH)$, where $O(N)$ is the work at each level and H is the # of levels. Since the pivot always lands at least 10% from either edge, H is approximately $\log_{10/9} N = O(\log N)$. Thus, overall runtime is $O(N \log N)$.

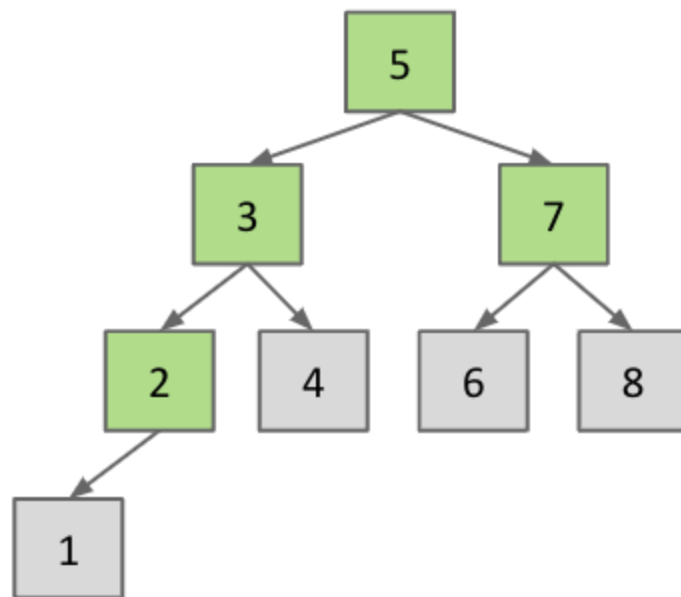


10% case partition work at each level.

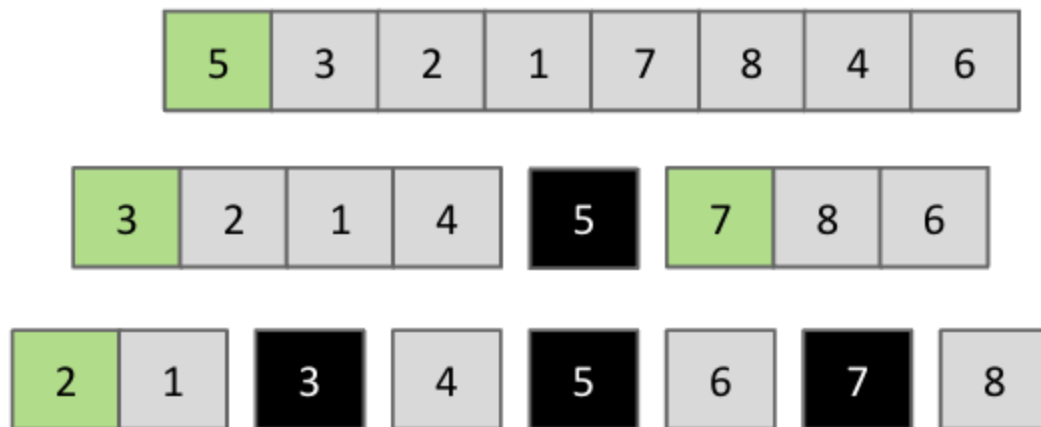
In other words, even if you are not lucky enough to have a pivot that is near the middle, if you can at least have a pivot that is always 10% from the edge, the runtime will be $O(N \log N)$.

Quicksort's Performance Argument #2: Quicksort is BST Sort

From another lens, Quicksort can be seen as a form of BST sort. This is because the `compareTo` calls that Quicksort employs between each element and the pivot element in each partition is the same as the `compareTo` calls performed in BST insert.



BST insert



Quicksort partitions on the “same” BST elements as above.

Since random insertion into a BST takes $O(N \log N)$ time, Quicksort can be argued to have similar performance.

Empirical Quicksort Runtime

Empirically, for N items, Quicksort uses an average of $\sim 2N \ln N$ compares to complete (across 10,000 trials with $N = 1000$). For more information, [check out this link](#).

Avoiding the Worst Case of Quicksort

As you can see from above, the performance of Quicksort (both in terms of order of growth and in constant factors) depends critically upon **how you select the pivot, how you partition around the pivot, and other optimizations that you can add to speed things up.**

Given certain conditions, Quicksort can result in $\Theta(N^2)$, which is much worse than $\Theta(N \log N)$. Some of these conditions include **bad ordering**, where the array is already in sorted order (or almost sorted order), and **bad elements**, where the array has all duplicates.

30.4 Summary

Insertion Sort Sweet Spots. We concluded our discussion of insertion sort by observing that insertion sort is very fast for arrays that are almost sorted, i.e. that have $\Theta(N)$ inversions. It is also fastest for small N (roughly $N \leq 15$).

Partitioning. Partitioning an array on a pivot means to rearrange the array such that all items to the left of the pivot are $\leq$ the pivot, and all items to the right are $\geq$ the pivot. Naturally, the pivot can move during this process.

Partitioning Strategies. There are many particular strategies for partitioning. You *are* expected to know how Quicksort partitioning works.

Quicksort. Partition on some pivot. Quicksort to the left of the pivot. Quicksort to the right.

Quicksort Runtime. Understand how to show that in the best case, Quicksort has runtime $\Theta(N \log N)$, and in the worse case has runtime $\Theta(N^2)$.

Pivot Selection. Choice of partitioning strategy and pivot have profound impacts on runtime. Two pivot selection strategies that we discussed: Use leftmost item and pick a random pivot. Understand how using

leftmost item can lead to bad performance on real data.

Randomization. Accept (without proof) that Quicksort has on average $\Theta(N \log N)$ runtime. Picking a random pivot or shuffling an array before sorting (using an appropriate partitioning strategy) ensures that we're in the average case.

Quicksort properties. For most real world situations, quicksort is the fastest sort.

30.5 Exercises

Factual

1. Suppose we have a sorted array in Java. What is the best approach for finding a particular item in the array: binary search or linear iteration?
2. Repeat the above exercise for a sorted array stored on physical tape.
3. Given the array [90, 50, 20, 30, 40, 10, 60, 50, 30], what would be the best pivot?

- Problem 1
- Problem 2
- Problem 3

Procedural

1. If we have 10 items to sort, how many compares will it take to selection sort all of the items?
2. What is the depth of the quicksort recursive tree in the best and worst case? Give an exact answer, not a theta bound.

- Problem 1
- Problem 2

Metacognitive

1. Assume that we always pick the leftmost item as our pivot for quicksort. Identify the three types of arrays that will cause a worst-case runtime. For partitioning, assume that we partition into two arrays: items less than or equal to the pivot, and items greater than the pivot.

- Problem 1

30. Quicksort